Distributed Locks

one worker at a time, even when things pause

Watch the 4-min video

In one line: a distributed lock lets one process at a time do something across machines, but a lock with an expiry can be lost during a pause without the holder knowing, so for correctness pair it with fencing tokens checked by the protected resource, and prefer consensus-backed stores.

1 It worked for months

Only one worker should process the nightly payout job, so you add a lock in Redis. It works for months, until one night a worker pauses, its lock expires, another worker takes the lock and starts paying out, and then the first one wakes up and continues. Every merchant is paid twice.

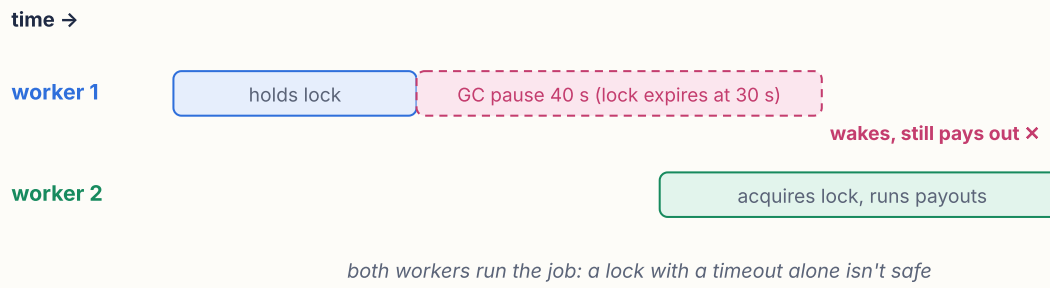


Figure 1. The first worker's lock expired during its pause, but the worker didn't know.

2 Two reasons to lock

Efficiency: avoid doing the same work twice (rebuilding a cache, sending a report). If the lock occasionally fails, you waste some work. **Correctness:** two holders would corrupt data or move money twice. These need a much stronger design. Always ask which one you need.

3 Locks need expiry, and expiry is the danger

A lock without an expiry is held forever if its owner crashes, so locks get a **lease** or TTL. But the holder can be paused by GC, a VM migration, or a slow disk past its TTL, and it can't detect that from inside: it might check 'do I still hold the lock?' and then pause right after the check. Clock jumps make TTLs even less reliable.

4 Fencing tokens

The safe fix moves the check to the **resource**. Each time the lock is granted, the lock service returns a number that only grows: a **fencing token**. The worker sends it with every write, and the resource remembers the highest token seen and rejects anything lower. A paused ex-holder's writes are refused.

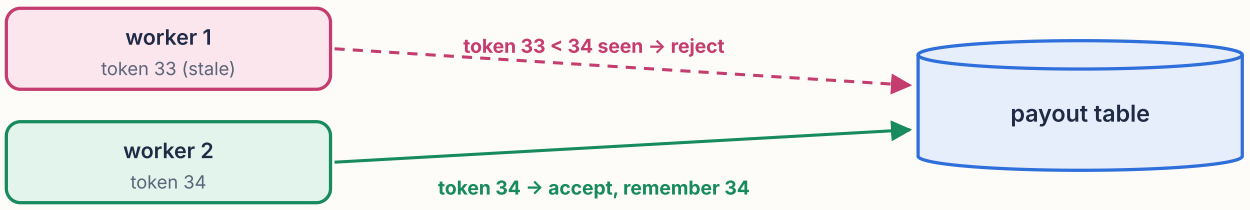


Figure 2. The resource, not the worker, enforces who may write.

WORKED EXAMPLE: PAYOUTS WITH AN ETCD LOCK

Three workers can run the nightly payout job. Each tries to acquire an etcd lock with a 30-second lease, and the winner gets a token: the lock key's revision, say 34. It keeps the lease alive while working, and writes each payout with `UPDATE payouts SET … WHERE batch_id = $1 AND last_token < 34`, storing 34. If it pauses and a new holder gets revision 35, the old worker's updates match no rows. Each payout also has an idempotency key, so a retried payout is never sent twice.

5 Which lock service

Redis SET NX PX	etcd / ZooKeeper	Database row lock
fast; fine for efficiency locks (avoid duplicate work)	consensus-backed; leases + revision as token	SELECT ... FOR UPDATE or advisory lock, in one DB

Figure 3. Choose by need: efficiency or correctness.

A single Redis node loses its locks on failover; the multi-node **Redlock** algorithm is debated because it depends on timing assumptions. Use Redis locks for efficiency. For correctness, use etcd or ZooKeeper (consensus, leases, monotonically increasing revisions) plus fencing, or let the database enforce it with a row lock or unique constraint. Often the best lock is none: make the operation idempotent.

COMMON MISTAKES

- Releasing a lock you no longer own: always check the value (a random owner ID) before deleting.
- TTL shorter than the job, with no renewal.
- Trusting a lock for money movement without fencing or idempotency.
- Locking on one Redis node and calling it highly available.

6 Key terms

Distributed lock: mutual exclusion across processes and machines

Fencing token: increasing number checked by the resource

Efficiency vs correctness lock: avoid duplicate work vs prevent corruption

Lease / TTL: lock expires unless renewed

Redlock: Redis multi-node lock algorithm, debated

7 Interview questions

Q1. Why must a distributed lock have a timeout? BASIC

If the holder crashes, the lock would never be released. A TTL or lease frees it, but then a slow or paused holder can lose it unknowingly.

Q2. What is a fencing token? BASIC

A strictly increasing number issued with each lock grant; the protected resource rejects requests with a token lower than the highest it has seen.

Q3. Design a lock so only one worker processes a payout batch. SCENARIO

Acquire a lease in etcd or ZooKeeper, use the revision as a fencing token, check it in the database update, renew the lease while working, and make each payout idempotent.

Q4. Your cache rebuild runs on 10 pods and hammers the DB. Do you need a strong lock? SCENARIO

No, an efficiency lock is enough: Redis SET NX with a TTL. An occasional duplicate rebuild is harmless.

Q5. Why isn't checking the lock right before writing enough? TRICKY

The process can pause between the check and the write, and the lock can expire and be taken by someone else in that gap.

Q6. What's the criticism of Redlock? TRICKY

It relies on bounded clock drift and pauses for safety and offers no fencing token, so under GC pauses or clock jumps two clients can believe they hold the lock.

Next: SD-H-014 · Gossip Protocols backendroadmap.com